

Git and GitHub in one shot

1. Difference between centralized and distributed version control?

- CVC: All version history and files are stored on a central server. Does't have local storage to store copy of the data pushed to central repository. Developers pull the latest version and commit changes directly to the central repository.
- DVCS: Every developer's machine has a full local copy of the entire repository, including the complete version history. Developers can commit, revert, and review history locally, even without internet access. Changes are later pushed to or pulled from a central/shared repository to collaborate with others.

Git init - is to initialize a empty directory in git

```
sagarbhavani@Sagars-MacBook-Air Ec2 keys % mkdir git-main
sagarbhavani@Sagars-MacBook-Air Ec2 keys % cd git-main
sagarbhavani@Sagars-MacBook-Air git-main % ls
sagarbhavani@Sagars-MacBook-Air git-main % touch file.txt
sagarbhavani@Sagars-MacBook-Air git-main % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-main % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /Users/sagarbhavani/Desktop/ Ec2 keys/git-main/.git/
sagarbhavani@Sagars-MacBook-Air git-main % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-main % ls -a
.      ..      .git      file.txt
sagarbhavani@Sagars-MacBook-Air git-main %
```

Git status - gives information about which branch we are in, how many commits we have made, and how many tracked or untracked files are there.

```
sagarbhavani@Sagars-MacBook-Air git-main % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-main % ls -a
.      ..      .git      file.txt
sagarbhavani@Sagars-MacBook-Air git-main % git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file.txt

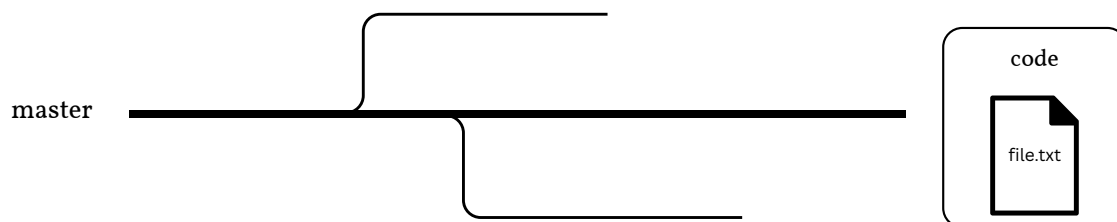
nothing added to commit but untracked files present (use "git add" to track)
```

Git Branches

What is Branch?

→ the branch is created from the main branch

Suppose we have run git init and created a branch which will be the main or master branch in which we have written the code we don't want to change the main code but want to perform some experiments in that case we create separate branches from main branch so a copy of the main code will also be in the new branch.



git reset: Deletes a commit from branch history.

Git revert: creates a new commit to undo changes of last commit.

git rebase: moves commits from one branch onto another to keep history clean.

Commands for git and git branches

- git branch checkout -b <new branch> - to checkout from one branch and create a new branch.
- git checkout - to switch between branches

- git add <filename> - Used to stage the untracked file
- git add . - to add all tracked and untracked files
- git commit -m <commit message> - Used to track the staged files after being added by git add.
- Git log - to get Information about author who commits

```
sagarbhavani@Sagars-MacBook-Air git-repo % touch new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    new.txt

nothing added to commit but untracked files present (use "git add" to track)
sagarbhavani@Sagars-MacBook-Air git-repo % git add new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   new.txt
```

```
sagarbhavani@Sagars-MacBook-Air git-main % git branch
sagarbhavani@Sagars-MacBook-Air git-main % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-main % git checkout -b dev
Switched to a new branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-main % git status
On branch dev

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
sagarbhavani@Sagars-MacBook-Air git-repo % git commit -m "added new.txt"
[master 6f6396c] added new.txt
  Committer: Sagar Bhavani <sagarbhavani@Sagars-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
nothing to commit, working tree clean
sagarbhavani@Sagars-MacBook-Air git-repo %
```

- once I have committed things to the branch then I can switch between branches using git checkout <branch-name>

```
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
nothing to commit, working tree clean
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout dev
Switched to branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-repo %
```

```
sagarbhavani@Sagars-MacBook-Air git-repo % git branch
* dev
  master
sagarbhavani@Sagars-MacBook-Air git-repo %
```

- If I delete a file and want to restore it again

Here a Question arises how it will restore it back if we delete the file from trash as well?

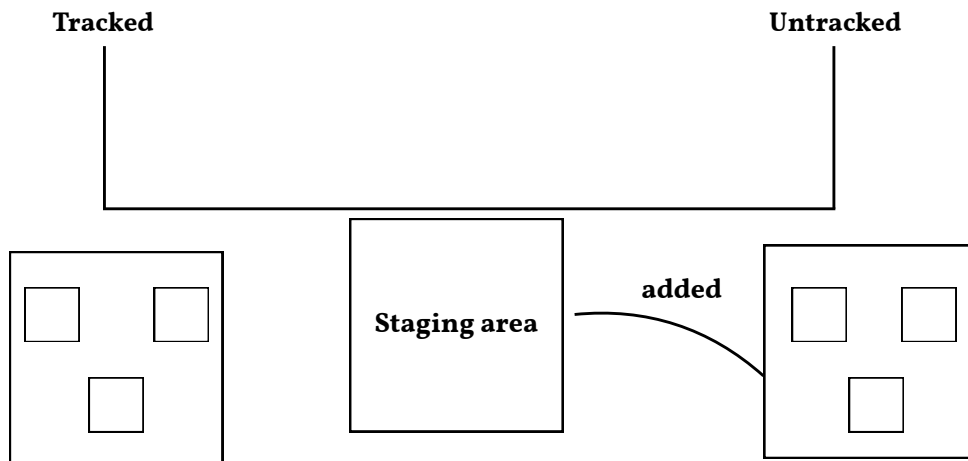
-> Because it was stored in git it will get back the file even if you delete it.

```
sagarbhavani@Sagars-MacBook-Air git-repo % cat file.txt
print ("Hello world")
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch dev
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
sagarbhavani@Sagars-MacBook-Air git-repo % rm file.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch dev
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    file.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Suppose I have added the file to staging area and i want to unstage it and keep it untracked.



```
sagarbhavani@Sagars-MacBook-Air git-repo % git add .
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
nothing to commit, working tree clean
sagarbhavani@Sagars-MacBook-Air git-repo % vi new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git add .
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   new.txt

sagarbhavani@Sagars-MacBook-Air git-repo % git restore --staged new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   new.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

```
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   file1.text
        deleted:    new.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    file1.text
```

To permanently delete a file

```
sagarbhavani@Sagars-MacBook-Air git-repo % rm new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    new.txt

no changes added to commit (use "git add" and/or "git commit -a")
sagarbhavani@Sagars-MacBook-Air git-repo % git add new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    new.txt

sagarbhavani@Sagars-MacBook-Air git-repo % git commit -m "deleted new.txt"
[master 9b42eb1] deleted new.txt
Committer: Sagar Bhavani <sagarbhavani@Sagars-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 1 deletion(-)
delete mode 100644 new.txt
```

- Git revert - Creates a new commit that inverts the changes of a previous commit.

```
sagarbhavani@Sagars-MacBook-Air git-repo % git status
On branch master
nothing to commit, working tree clean
sagarbhavani@Sagars-MacBook-Air git-repo % git log
commit 2eed059d5c5044ff652be23de7483213fc8a620 (HEAD -> master)
Author: Sagar Bhavani <sagarbhavani@Sagars-MacBook-Air.local>
Date:   Fri May 2 23:48:17 2025 +0530

    deleted new.txt
```

```
sagarbhavani@Sagars-MacBook-Air git-repo % git revert 5ab7ac3e6981446b41112c9f37ea341deb6e54c7
[dev 92a68bf] Revert "deleted new file"
Committer: Sagar Bhavani <sagarbhavani@Sagars-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 1 insertion(+)
create mode 100644 new.txt
```

If I checkout and create a new branch from master it will have the changes of master and if I checkout and create a new branch from dev it will dev changes

```
sagarbhavani@Sagars-MacBook-Air git-repo % git branch
* dev
  master
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout -b new-dev
Switched to a new branch 'new-dev'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout dev
Switched to branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout master
Switched to branch 'master'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt      new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout dev
Switched to branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout master
Switched to branch 'master'
sagarbhavani@Sagars-MacBook-Air git-repo % git branch
  dev
* master
  new-dev
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout -b new_prod
Switched to a new branch 'new_prod'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt      new.txt
```

If I checkout and create a new branch from master it will have the changes of master and if I checkout and create a new branch from dev it will dev changes

```
sagarbhavani@Sagars-MacBook-Air git-repo % git branch
* dev
  master
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout -b new-dev
Switched to a new branch 'new-dev'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout dev
Switched to branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout master
Switched to branch 'master'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt      new.txt
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout dev
Switched to branch 'dev'
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout master
Switched to branch 'master'
sagarbhavani@Sagars-MacBook-Air git-repo % git branch
  dev
* master
  new-dev
sagarbhavani@Sagars-MacBook-Air git-repo % git checkout -b new_prod
Switched to a new branch 'new_prod'
sagarbhavani@Sagars-MacBook-Air git-repo % ls
file.txt      new.txt
```